النموذج الموحّد الثاني: 12 فئة، بيانات من عدة مصادر، وتدريب أطول

ما فعلناه، ولماذا، والنتائج الصادقة (بما فيها ما لم ينجح)

12	+0.13	18.7 FPS	6.4 h
فئة في ملف .pt واحد	تحسّن AP@0.5 على الفئات الأربع المشتركة مقارنة بالنموذج القديم	على المعالج (CPU) بصورة 512	25 جولة تدريب

الخلاصة في ثلاث جمل

1. **أفضل بوضوح من النموذج القديم:** على صور لم يرها أي نموذج، ارتفعت الدقة في الفئات الأربع المشتركة (شخص، سيارة، قطة، كلب) من 0.534 إلى **0.667** (AP@0.5)، وهو تحسّن في كل فئة منها.
2. **يكتشف 12 فئة** بدل 5، منها المسيّرة (drone) التي لا يعرفها أي نموذج جاهز.
3. **لكنه أقل من YOLO11n الجاهز في فئات COCO** بنحو 0.08 (AP@0.5)، وعلى الأرجح لأن التدريب على مجموعة جزئية من COCO أنسى النموذج جزءاً من معرفته الأصلية (لم نفصل هذا السبب بتجربة). الجاهز لا يعرف المسيّرة.

المحتويات

1. ما الذي تغيّر عن النموذج الأول
2. البيانات من عدة مصادر
3. التدريب
4. طريقة التقييم العادل
5. النتائج
6. أمثلة كشف (ونقاط الفشل)
7. تحفظات يجب ذكرها
8. التشغيل بكاميرا الهاتف
9. ما تعلمناه، وخطوات مقترحة

1. ما الذي تغيّر عن النموذج الأول

طلب الأستاذ نموذجاً يكتشف أشياء أكثر. وقلتَ لنا ألّا نلتزم بمصدر واحد (COCO)، وأن نزيد عدد الجولات. هذه هي التغييرات:

الجانب	النموذج القديم (المرحلة 2)	النموذج الجديد
الشبكة	YOLOv5n (1.76 مليون معامل)	**YOLO11n** (بنية أحدث، تبدأ من أوزان COCO الجاهزة)
الفئات	5: شخص، سيارة، قطة، كلب، مسيّرة	**12:** شخص، دراجة، دراجة نارية، سيارة، حافلة، شاحنة، طائرة، طائر، قطة، كلب، حصان، مسيّرة
مصادر البيانات	COCO + 700 صورة مسيّرات	**900** COCO + **صورة مسيّرات جديدة** من قسم آخر في Seraphim + **250 صورة خلفية**
عدد صور التدريب	2480	**5106** (+ 444 للتحقق)
حجم الصورة	416	**512** (المسيّرات صغيرة)
الجولات	30	25 (لكن على بيانات ضعف حجمها تقريباً، فالمجموع أكبر)
وقت التدريب	~2 ساعة	**6.4 ساعة** على CPU

لماذا 25 جولة وليس 100؟

الجهاز هنا بلا GPU: كل جولة تأخذ 15 دقيقة. والنموذج يبدأ من أوزان جاهزة، فيقترب من أفضل أداء خلال عشرات الجولات لا مئاتها. 25 جولة هي ما يتسع له الجهاز في ساعات معقولة. تدريب أطول على GPU قد يعطي نتيجة أفضل، لكنني لم أجرّبه.

2. البيانات من عدة مصادر

المصدر	ماذا أخذنا	صور
COCO train2017	11 فئة، حوالي 400 صورة لكل فئة. كل جسم من فئاتنا في الصورة معلَّم، مهما صغر.	4400
COCO (صور خلفية)	صور ليس فيها أي فئة من فئاتنا. تعلّم النموذج ألّا يكتشف شيئاً حيث لا يوجد.	250
Seraphim (Hugging Face، CC-BY-4.0)	مسيّرات من القسم train، 900 صورة صالحة. أُعيدت تسمية فئتها إلى 11.	900

المجموع 5550 صورة، قُسمت إلى **5106 للتدريب** و**444 للتحقق** (بذرة ثابتة 42). عدد المربعات في التدريب:

الفئة	شخص	دراجة	دراجة نارية	سيارة	حافلة	شاحنة	طائرة	طائر	قطة	كلب	حصان	م
مربعات التدريب	8883	992	1126	3858	798	1279	715	768	468	594	891	

التسميات الزائفة (Pseudo-labeling) لصور المسيّرات

صور المسيّرات تضع علامة على المسيّرة فقط، لكن فيها أشخاص (يد تمسك جهاز التحكم) وسيارات بلا علامات. لو تدرّب النموذج عليها كما هي لتعلّم أن "هذا الشخص خلفية". لذلك مررنا هذه الصور على **YOLO11m الجاهز** (بثقة ≥ 0.5)، فأضاف **251 علامة في 189 صورة**.

> **احتياطات ضد أخطاء المساعد**
>
> - استبعدنا فئتَي **طائر** و**طائرة** من هذه الخطوة: المساعد قد يسمّي المسيّرة نفسها "طائراً" أو "طائرة".
> - تجاهلنا أي مربع يتداخل مع مربع المسيّرة (IoU أو تغطية عالية).
> - راجعنا عيّنة بصرياً: العلامات المضافة أشخاص وأيدٍ فعلاً، ولم نجد مسيّرة مسمّاة بفئة أخرى.
>
> مع ذلك تبقى هذه علامات آلية فيها ضجيج محتمل، وتُحتسب كأنها صحيحة أثناء التدريب.

`scripts/build_dataset_v2.py`

```
PSEUDO_CLASSES = [c for c in COCO_CLASSES if c not in ("bird", "airplane")]    # لا نسمح للمساعد بتسمية المسيّرة طائراً

for r in predictions():                                    # نمرّر الصور على YOLO11m، 8 صور في كل مرة
    drones = [مربعات المسيّرة الموجودة في العلامات]
    for box, cls in zip(r.boxes.xywhn, r.boxes.cls):
        if overlaps_a_drone(box, drones):
            continue                                       # يتداخل مع مسيّرة ← غالباً هو المسيّرة نفسها
        new.append(f"{ours[cls]} {box}")                   # نضيف العلامة بصيغة YOLO
    open(label_path, "a").write(new)                       # "a" ← نضيف إلى علامات المسيّرة، لا نمسحها
```

مشكلة واجهتها وأصلحتها

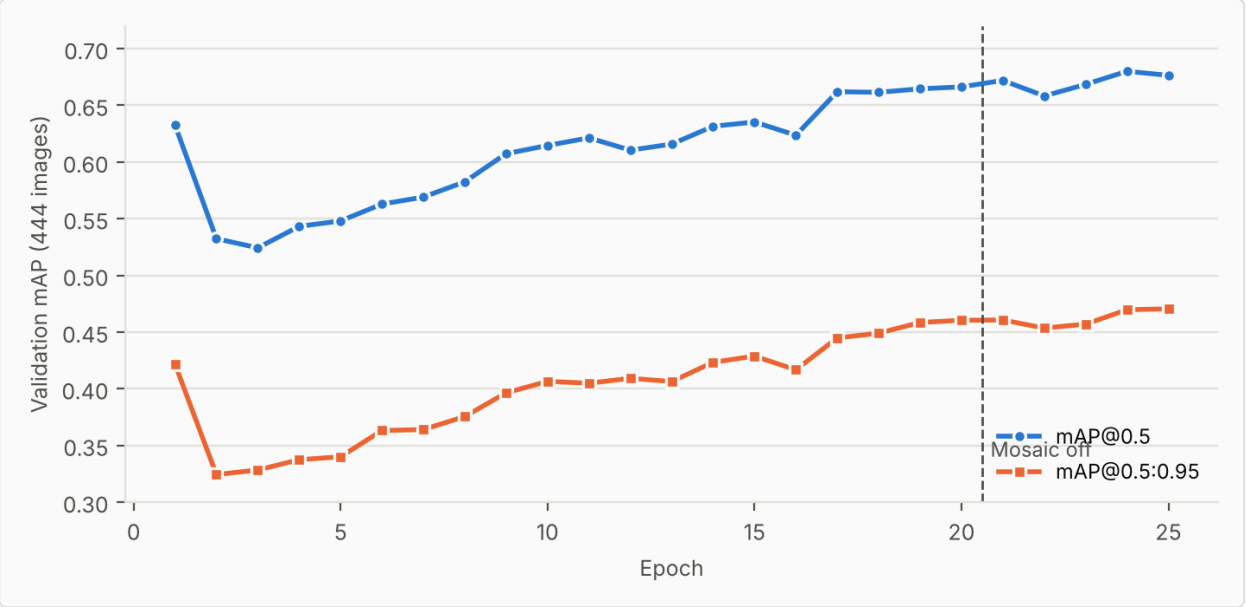
أول مرة مرّرتُ قائمة 900 مسار دفعة واحدة، فحاولت المكتبة وضع كل الصور في دفعة واحدة وطلبت **23.6 GB** من الذاكرة وفشلت. كما أن الخيار `batch=8` لم يُطبَّق في هذه النسخة. الحل: تقسيم القائمة بنفسي إلى أجزاء من 8 صور.

3. التدريب

`scripts/train_v2.py`

```
model = YOLO("yolo11n.pt")                          # نبدأ من أوزان COCO الجاهزة (تعلّم بالنقل)
model.train(data="data/v2_dataset/data.yaml",
            epochs=25, imgsz=512, batch=16,
            device="cpu", workers=4,
            close_mosaic=5,                         # نوقف دمج 4 صور (Mosaic) في آخر 5 جولات
            cache="ram",                            # الصور المصغّرة في الذاكرة: أسرع على المعالج
            seed=0)
# المُحسِّن يُختار تلقائياً: AdamW بمعدل تعلم 0.000625
```

كيف تطوّرت الدقة أثناء التدريب؟



mAP على مجموعة التحقق (444 صورة) بعد كل جولة. الخط المتقطع: بداية إيقاف Mosaic (الجولة 21).

لماذا بدأت الدقة عالية، ثم هبطت، ثم ارتفعت؟ (تفسير مرجّح، لم نختبره)

بعد الجولة الأولى كان **0.633** = mAP@0.5، ثم هبط إلى **0.524**، ثم ارتفع تدريجياً إلى **0.676**.

- في الجولة الأولى معدل التعلم صغير جداً (فترة إحماء)، فالنموذج ما زال قريباً من أوزان COCO الجاهزة. ومجموعة التحقق عندنا من صور COCO train2017 التي رأتها تلك الأوزان، فتبدو الدقة عالية.
- عندما يرتفع معدل التعلم يتغيّر النموذج بسرعة ويفقد جزءاً من معرفته الأصلية، ثم يتحسن مع انخفاض معدل التعلم وتعلّمه للمسيّرة.
- هذا ما يُعرف بالنسيان (catastrophic forgetting)، وهو التفسير الأرجح لكون النموذج النهائي أقل من الجاهز على فئات COCO (الفصل 5). لم نفصل هذا السبب عن غيره بتجربة.

درس عملي: انقطاع الجهاز

أُعيد تشغيل الجهاز مرات أثناء الأعمال الطويلة، فتُقتل العمليات الخلفية. الجولة لا تُحفظ إلا في نهايتها، فخسرنا أول محاولتين من هذا التدريب (بين 4 و12 دقيقة لكل منهما). ما نفع: `train_v2.py` يستأنف من آخر نقطة محفوظة، وبقاء الجلسة نشطة أثناء التدريب. يبدو أن الجهاز يُعاد تشغيله حين تبقى الجلسة خاملة (هذا استنتاج من تكرار الحدث، لم أتحقق منه)، فبقيتُ في انتظار نشط حتى انتهى التدريب.

4. طريقة التقييم العادل

أخطر خطأ في مقارنة النماذج أن يُختبر نموذج على صور رآها أثناء التدريب. لذلك بنينا **مجموعتَي اختبار لم يرهما أي نموذج في المقارنة**:

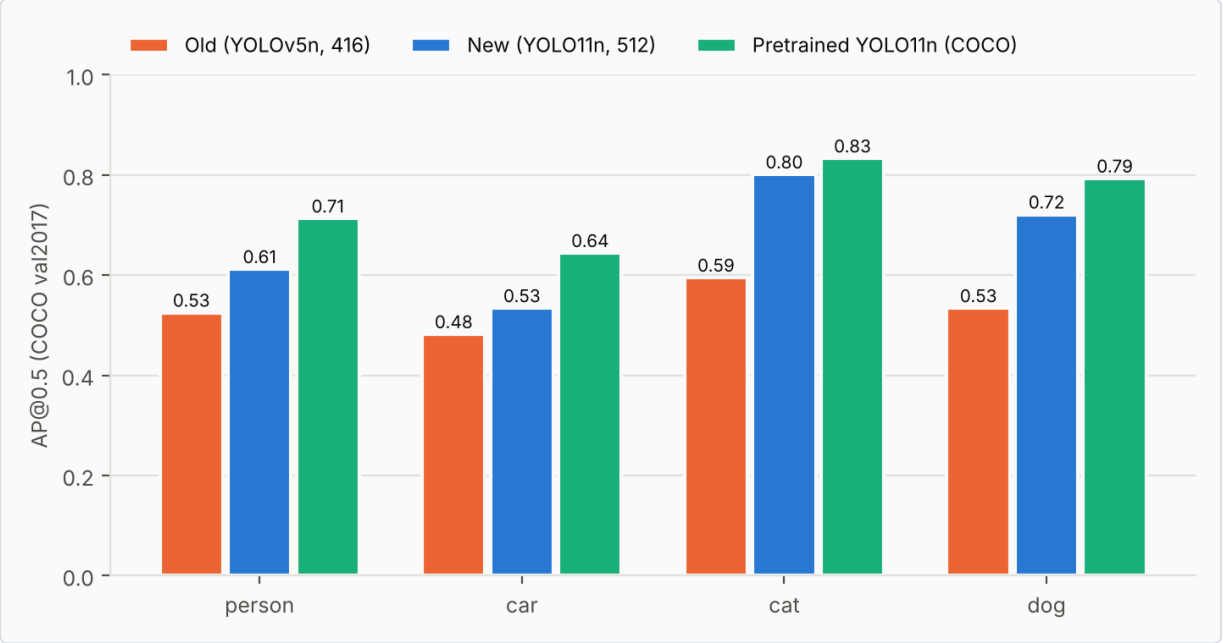
المجموعة	الصور	المصدر	لماذا هي عادلة؟
COCO اختبار	440	COCO **val2017**، الفئات الـ 11	كل النماذج (القديم، الجديد، الجاهز) تدرّبت على train2017 فقط
مسيّرات اختبار	300	قسم test في Seraphim، بعد استبعاد الصور الـ 700 التي استعملها النموذج القديم	لم تُستعمل في تدريب النموذج القديم ولا الجديد

- **كل نموذج يُقيَّم على الفئات التي يعرفها فقط**، فلا يُعاقَب نموذج على فئة لم يتدرّب عليها. الفئات تُطابَق بالاسم.
- **الفئات الأربع المشتركة** (شخص، سيارة، قطة، كلب) هي التي نقارن بها القديم بالجديد، لأن القديم لا يعرف غيرها.
- صور المسيّرات تحمل علامة المسيّرة فقط (بدون تسميات زائفة) حتى لا يتأثر التقييم بعلامات آلية.
- نفس الكود والمقياس (`torchmetrics`، حد ثقة 0.001) لكل النماذج: `scripts/evaluate_v2.py`.
- الأرقام مقياس **AP@0.5** (قبول المربع إذا كان IoU ≥ 0.5) و**AP@0.5:0.95** (المقياس الأصعب).

5. النتائج

أ) القديم مقابل الجديد، على الفئات الأربع المشتركة (COCO val2017)

الفئة	AP@0.5 قديم	AP@0.5 جديد	الفرق	AP@0.5:0.95 قديم	AP@0.5:0.95 جديد	الفرق
شخص (person)	0.525	0.612	+0.087	0.272	0.377	+0.105
سيارة (car)	0.482	0.534	+0.052	0.249	0.345	+0.096
قطة (cat)	0.595	0.801	+0.206	0.341	0.597	+0.256
كلب (dog)	0.534	0.721	+0.187	0.344	0.582	+0.238
المتوسط	0.534	**0.667**	**+0.133**	0.301	**0.475**	**+0.174**



AP@0.5 لكل فئة على صور COCO val2017 (440 صورة). الأعمدة الناقصة = النموذج لا يعرف الفئة.

النتيجة الأولى (واضحة): الجديد أفضل من القديم في كل فئة

تحسّن AP@0.5 للشخص +0.09، والسيارة +0.05، والقطة +0.21، والكلب +0.19. والمتوسط من 0.534 إلى 0.667، وAP@0.5:0.95 من 0.301 إلى 0.475. الفرق كبير ومتسق في الفئات الأربع، فهو ليس ضجيجاً عشوائياً.

5. (تابع) كل الفئات، والمسيّرة، والسرعة

ب) الجديد مقابل YOLO11n الجاهز، على الفئات الـ 11 (COCO val2017)

الفئة	جديد AP@0.5 (512)	جاهز AP@0.5 (640)	الفرق	AP@0.5:0.95 جديد	AP@0.5:0.95 جاهز
شخص (person)	0.612	0.713	-0.101	0.377	0.470
دراجة (bicycle)	0.481	0.565	-0.084	0.296	0.355
دراجة نارية (motorcycle)	0.675	0.765	-0.090	0.420	0.514
سيارة (car)	0.534	0.644	-0.110	0.345	0.435
حافلة (bus)	0.791	0.846	-0.055	0.654	0.754
شاحنة (truck)	0.394	0.454	-0.060	0.252	0.330
طائرة (airplane)	0.789	0.834	-0.045	0.589	0.701
طائر (bird)	0.573	0.624	-0.051	0.420	0.493
قطة (cat)	0.801	0.833	-0.032	0.597	0.641
كلب (dog)	0.721	0.793	-0.072	0.582	0.623
حصان (horse)	0.722	0.829	-0.107	0.478	0.631
المتوسط (11 فئة)	**0.645**	**0.718**	-0.073	0.455	0.541

النتيجة الثانية: الجاهز أفضل في فئات COCO، وهذا مهم أن تفهمه

YOLO11n الجاهز أدق من نموذجنا في **كل** فئة من الفئات الـ 11: متوسط AP@0.5 هو 0.718 مقابل 0.645 (فرق 0.073). السبب الأرجح أنه تدرّب على 118 ألف صورة طوال مئات الجولات، ونحن ضبطناه على 4400 صورة فقط، فنسي جزءاً مما يعرفه. هذا تفسير لم نختبره بتجربة تعزله عن غيره. **الجاهز لا يعرف المسيّرة**، فلكل منهما ميزة: من يحتاج المسيّرة يختار نموذجنا، ومن يحتاج فئات COCO فقط يستعمل الجاهز.

ج) المسيّرة (300 صورة لم يرها أي نموذج)

النموذج	AP@0.5	AP@0.5:0.95
قديم (YOLOv5n، 416)	0.822	0.415
جديد (YOLO11n، 512)	0.812	**0.491**
جديد (YOLO11n، 640)	0.794	0.473
YOLO11n الجاهز	لا يعرف الفئة	لا يعرف الفئة

AP@0.5 متقارب جداً بين القديم والجديد (0.822 و0.812)، والفرق أقل من الضجيج المتوقع. أما AP@0.5:0.95 فالجديد أفضل بوضوح (0.415 إلى 0.491)، أي أن المربعات أدق على المسيّرة، حتى لو كان العثور عليها بنفس الكفاءة.

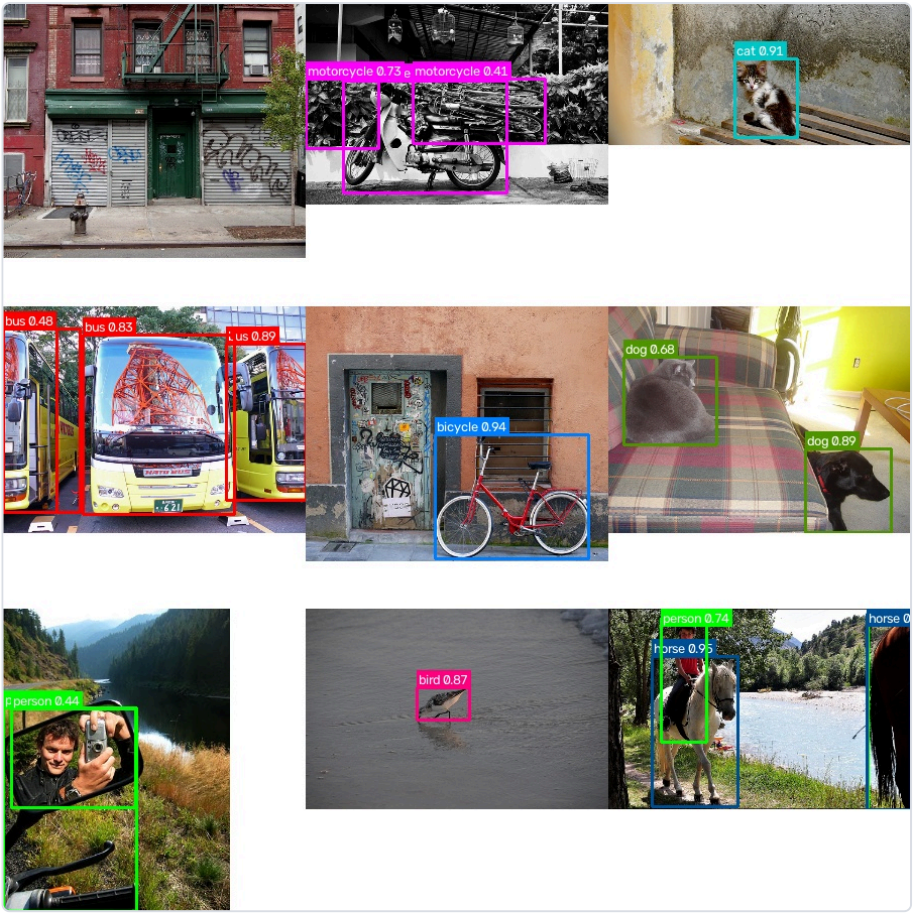
د) السرعة وحجم الصورة

النموذج	حجم الصورة	ملّي ثانية / صورة (الاستدلال فقط)	تقريبي FPS	عدد الفئات
قديم (YOLOv5n)	416	40	25	5
جديد	512	56	18	12
جديد	640	76	13	12
YOLO11n الجاهز	640	77	13	11

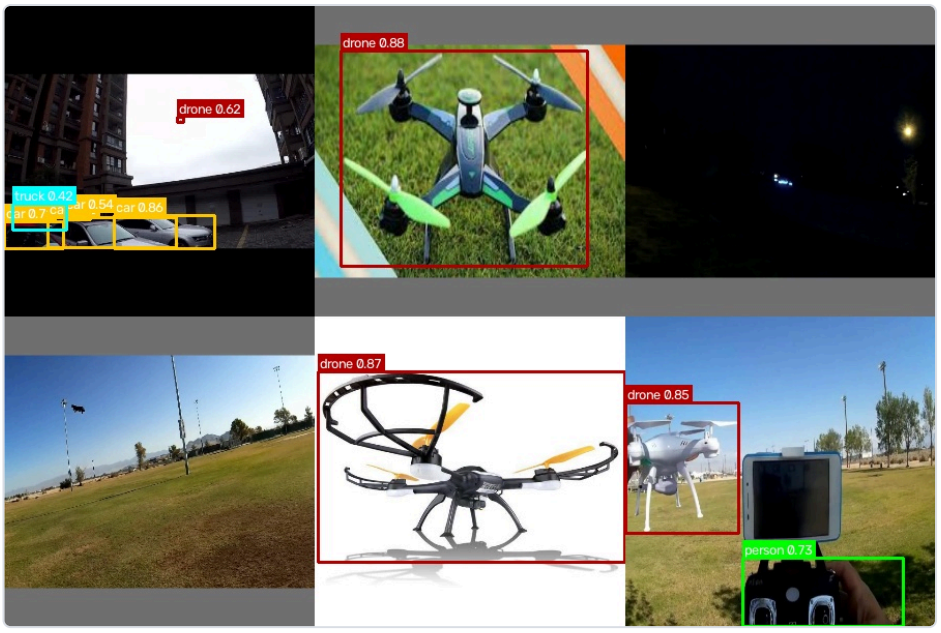
- **التكبير إلى 640 لا يفيد** هنا: AP@0.5 على الفئات الـ 11 متساوٍ تقريباً (0.645 مقابل 0.645 عند 512)، وAP@0.5:0.95 أقل (0.441 مقابل 0.455)، وهو أبطأ. نبقى على 512.
- في فيديو المشاة، بالسكربت الحي: **18.7 FPS** على المعالج (يشمل قراءة الفيديو ورسم المربعات).
- الجديد أبطأ من القديم (56 مقابل 40 ملّي ثانية للصورة)، ثمن الحجم الأكبر والبنية الأحدث.

6. أمثلة كشف (ونقاط الفشل)

الصور مأخوذة بفواصل منتظمة من مجموعات الاختبار (غير منتقاة)، بحد ثقة 0.35.



صور COCO اختبار. الكشف صحيح في معظمها. في الصورة العلوية اليسرى (مبنى) لا شيء من فئاتنا، فلا كشف، وهذا صحيح.



صور المسيّرات. الكشف ناجح على المسيّرات الكبيرة. **الفشل:** مسيّرة صغيرة جداً فوق الحقل (الصف الثاني، الصورة اليسرى) لم تُكتشف، وكذلك مشهد ليلي مظلم (الصف الأول، الصورة اليمنى). وكل صور الاختبار فيها مسيّرة معلَّمة. من الكشوف الصحيحة: مسيّرة بعيدة جداً في أعلى الصورة (الصف الأول، الصورة اليسرى) بثقة 0.62.

أين يفشل النموذج؟

المسيّرات الصغيرة جداً والبعيدة، والمشاهد المظلمة. هذا متوقع: النموذج صغير (2.6 مليون معامل)، والصورة 512، ومعظم المسيّرات في التدريب أكبر من هذه.

7. تحفظات يجب ذكرها للأستاذ

- **تدريب واحد بـ seed واحد.** لا نعرف مدى تذبذب النتائج بين تدريب وآخر. الفرق الكبير (القديم مقابل الجديد، +0.13) يطمئن، لكن الفروق الصغيرة (مثل 0.822 مقابل 0.812 للمسيّرات) لا يُعتدّ بها.
- **مجموعة التحقق من COCO train2017**، أي من صور رأتها أوزان COCO الجاهزة. لذلك أرقام التحقق أثناء التدريب متفائلة. الأرقام المعتمدة هي أرقام الاختبار (val2017) فقط.
- **صور المسيّرات قد تتشابه** بين قسمي train وtest في مجموعة Seraphim (مأخوذة من 23 مجموعة، وربما فيها لقطات متتابعة من نفس الفيديو). لا نعرف إن كان الأمر كذلك. إن حدث، تكون أرقام المسيّرات متفائلة للنموذجين معاً.
- **التسميات الزائفة** (251 علامة) فيها ضجيج محتمل.
- **السرعة** قيست على CPU بأربعة أنوية، صورة واحدة في كل مرة.
- **مقارنة النموذج القديم بالجديد** لا تفصل أثر كل عنصر: الشبكة، والبيانات الأكثر، وحجم الصورة، والجولات تغيّرت كلها معاً. نعرف أن المجموع أفضل، ولا نعرف أي عنصر ساهم أكثر.
- **لم نُجرِ تجربة الإخفاء الجزئي** على هذا النموذج.

8. التشغيل بكاميرا الهاتف

```
cd "C:\Users\chems\real time humain detection\real_time-_object_detection"
git pull
python -m pip install -r requirements.txt

# النموذج الجديد (12 فئة). السكربت يتعرف تلقائياً على نوع النموذج (YOLOv5 أو YOLO11)
python scripts/detect_realtime.py --weights models/v2_yolo11n_12cls.pt --source 1 --img-size 512 --conf 0.35
```

- **--img-size 512**: هو حجم التدريب، ونتائجه أفضل من 640 كما رأينا.
- **--conf 0.35**: إذا ظهرت كشوف خاطئة كثيرة ارفعه إلى 0.5، وإذا فاتته أشياء اخفضه إلى 0.25.
- لكل فئة لون مربع مختلف. اضغط `s` لحفظ صورة، و `q` للخروج.

> **جرّب هذه المشاهد وسجّل ملاحظاتك في التقرير**
>
> شخص (قريب وبعيد)، قطة أو كلب إن توفّر، دراجة، سيارات من النافذة. وإن كانت عندك مسيّرة (حتى لعبة) أو صورة لمسيّرة على شاشة، فجرّبها: من الأفضل أن تكتب هل اكتُشفت في مسافات مختلفة.

9. ما تعلمناه، وخطوات مقترحة

ما تعلمناه

1. **المزيد من البيانات المتنوعة وبنية أحدث يعطيان تحسّناً حقيقياً** (+0.13 AP@0.5 على الفئات المشتركة).
2. **الضبط على مجموعة جزئية يُنسي** النموذج بعض ما يعرفه. إن كانت الفئات كلها موجودة في COCO، فالنموذج الجاهز أفضل. التدريب يفيد حين نضيف ما لا يعرفه الجاهز (المسيّرة).
3. **التقييم العادل أهم من التدريب**: لو اختبرنا على صور التدريب لظهر النموذج الجديد أفضل مما هو.

خطوات تقلل نسيان فئات COCO (لم أجرّبها)

- معدل تعلم أقل، أو تجميد الجزء الأول من الشبكة (`freeze=10`) أثناء الضبط.
- بيانات COCO أكثر (آلاف الصور لكل فئة بدل 400)، وهذا يحتاج GPU لأن المعالج بطيء جداً.
- نموذج أكبر (YOLO11s) أو صورة أكبر، مع GPU.

أسئلة للمراجعة

1. لماذا قارنّا القديم بالجديد على الفئات الأربع المشتركة فقط؟

لأن القديم لا يعرف إلا 5 فئات. لو حسبنا متوسط الفئات الـ 11 لحصل القديم على صفر في الفئات التي لا يعرفها، وهذا ظلم لا يقيس جودة الكشف نفسها.

2. لماذا اختبرنا على COCO val2017 وليس على جزء من صور التدريب؟

كل النماذج تدرّبت على train2017. صور val2017 لم يرها أي نموذج، فنقيس قدرته على التعميم.

3. لماذا نموذج YOLO11n الجاهز أفضل منا في فئات COCO؟

تدرّب على 118 ألف صورة لمئات الجولات. نحن ضبطناه على 4400 صورة، فنسي جزءاً من معرفته (catastrophic forgetting).

4. لماذا استبعدنا "طائر" و"طائرة" من التسميات الزائفة لصور المسيّرات؟

المساعد قد يسمّي المسيّرة نفسها طائراً أو طائرة، فنعلّم النموذج الخطأ.

5. لماذا أضفنا صور خلفية بلا أي علامة؟

حتى يتعلم النموذج أن "لا شيء" إجابة صحيحة، فيقل الكشف الخاطئ في السماء الفارغة أو المشاهد العادية.

6. ما الدليل على أن التحسّن عن القديم ليس ضجيجاً؟

التحسّن كبير (+0.13) ومتسق في الفئات الأربع كلها. أما الفروق الصغيرة (مثل المسيّرات AP@0.5) فلا نستطيع الحكم عليها بتجربة واحدة.